

Aula 11 - Análise Estática e Qualidade de Código

Duração: 2 horas

Objetivos:

- Dominar o uso do SonarQube para análise estática
- Interpretar métricas de qualidade e technical debt
- Realizar refatoração baseada em dados objetivos

Parte Teórica (40 min)

1. O que o SonarQube Analisa?

Categoria	Exemplos de Métricas
Bugs	NullPointerException potencial
Code Smells	Métodos longos, duplicação de código
Vulnerabilidades	SQL Injection, hardcoded passwords
Cobertura	% de código coberto por testes

2. Métricas-Chave

- **Technical Debt:** Tempo estimado para corrigir problemas (ex: 2h 30min)
- **Maintainability Rating:** Nota de A a E (A=melhor)
- **Reliability:** Número de bugs
- **Security:** Número de vulnerabilidades

3. Quality Gates

Regras que definem quando o código está "pronto":

- Cobertura $\geq$ 80%
- 0 Vulnerabilidades Críticas
- Technical Debt < 1 dia

Parte Prática (1h20min)

Atividade 1: Configurando SonarQube Local

1. Inicie o SonarQube via Docker:

```
bash
```

```
docker run -d --name sonarqube -p 9000:9000 sonarqube:community
```

2. Acesse e faça login:

- URL: `http://localhost:9000`
- Credenciais padrão: `admin/admin`

3. Crie um novo projeto manualmente:

- Generate token (salve este token!)

Atividade 2: Analisando um Projeto Java

1. Adicione o plugin ao `pom.xml` :

```
xml
```

```
<plugin>
  <groupId>org.sonarsource.scanner.maven</groupId>
  <artifactId>sonar-maven-plugin</artifactId>
  <version>3.9.1.2184</version>
</plugin>
```

2. Execute a análise:

```
bash
```

```
mvn clean verify sonar:sonar \
  -Dsonar.projectKey=meu-projeto \
  -Dsonar.host.url=http://localhost:9000 \
  -Dsonar.login=SEU_TOKEN
```

3. Explore o dashboard:

- Identifique:
 - Maior code smell

- Arquivo com maior dívida técnica
- Método mais complexo (complexidade ciclomática)

Atividade 3: Refatoração Guiada

Código Problemático (exemplo):

java

```
public class FinanceUtils {  
    // Complexidade ciclomática = 8 (alta!)  
    public BigDecimal calcularImposto(BigDecimal valor, String tipo) {  
        if (valor == null) return BigDecimal.ZERO;  
  
        if (tipo.equals("ICMS")) {  
            return valor.multiply(new BigDecimal("0.17"));  
        } else if (tipo.equals("IPI")) {  
            if (valor.compareTo(new BigDecimal("10000")) > 0) {  
                return valor.multiply(new BigDecimal("0.10"));  
            } else {  
                return valor.multiply(new BigDecimal("0.05"));  
            }  
        } // ... mais 4-5 condições  
    }  
}
```

Exercício de Refatoração:

1. Extraia método para cada tipo de imposto
2. Use Strategy Pattern para eliminar if-else
3. Adicione testes para nova versão

Solução Parcial:

java

```
public interface ImpostoStrategy {  
    BigDecimal calcular(BigDecimal valor);  
}  
  
public class IcmsStrategy implements ImpostoStrategy {  
    public BigDecimal calcular(BigDecimal valor) {  
        return valor.multiply(new BigDecimal("0.17"));  
    }  
}
```

```

public class FinanceUtils {
    private Map<String, ImpostoStrategy> estrategias;

    public FinanceUtils() {
        this.estrategias = Map.of(
            "ICMS", new IcmsStrategy(),
            "IPI", new IpiStrategy()
        );
    }

    // Complexidade agora = 1
    public BigDecimal calcularImposto(BigDecimal valor, String tipo) {
        if (valor == null || !estrategias.containsKey(tipo)) {
            return BigDecimal.ZERO;
        }
        return estrategias.get(tipo).calcular(valor);
    }
}

```

Desafio em Grupo (30 min)

Contexto:

1. Divida-se em grupos de 3
2. Cada grupo:
 - Escolha 1 classe problemática do SonarQube
 - Planeje uma refatoração (15 min)
 - Implemente e verifique a melhoria no SonarQube

Critérios de Sucesso:

- Redução de $\geq 50\%$ na dívida técnica da classe
- Aumento na nota de maintainability (ex: C $\rightarrow$ B)

Encerramento

- Discussão: Qual métrica foi mais útil para suas decisões?
- Próxima aula: Documentação de Software

Tarefa:

- Refatorar 2 classes do projeto final usando insights do SonarQube
- Configurar um Quality Gate mínimo para o projeto

Recursos:

- [SonarQube Rules for Java](#)
- [Refactoring Guru - Strategy Pattern](#)

Transforme análise estática em código limpo! Dúvidas? 😊🚀